Pneumonia Detection from Chest X-Ray Images Using Deep Learning (PyTorch)

Mohamed Ouaddane

ESTIN

Project Report

October 2025

Abstract

This report presents a deep learning approach for pneumonia detection from chest X-ray images using PyTorch. Leveraging the Chest X-Ray (Pneumonia) dataset from Kaggle, we preprocess the data, implement a custom convolutional neural network (CNN) and ResNet18 architectures, and evaluate their performance. The models are trained to classify images as NORMAL or PNEUMONIA, addressing class imbalance. The custom CNN achieves a test accuracy of 97.96%, while ResNet18 achieves 92.67%. The study highlights the efficacy of deep learning in medical diagnostics and suggests directions for future improvements.

1 Introduction

Pneumonia is a critical respiratory condition, and early detection via chest X-ray imaging is vital for effective treatment. Deep learning, particularly convolutional neural networks (CNNs) and advanced architectures like ResNet, has shown promise in automating medical image analysis. This project aims to develop and compare deep learning models for binary classification of chest X-ray images into NORMAL and PNEUMONIA categories using the PyTorch framework. The report details the dataset, methodology, training, evaluation, and results, contributing to the field of AI-assisted medical diagnostics.

2 Dataset Description

The **Chest X-Ray (Pneumonia)** dataset, sourced from Kaggle (https://www.kaggle.com/paultimothymooney/chest-xray-pneumonia), contains 4,956 grayscale anterior-posterior chest X-ray images from Guangzhou Women and Children's Medical Center, China. The dataset is split into:

- **Train**: 3,861 images (1,083 NORMAL, 2,778 PNEUMONIA)
- **Validation**: 471 images (135 NORMAL, 336 PNEUMONIA)
- **Test**: 624 images (234 NORMAL, 390 PNEUMONIA)

Images are resized to 224×224 pixels and normalized to [0, 1]. The dataset exhibits a 2.4× class imbalance, addressed via weighted sampling.

3 Methodology

The methodology involves data preprocessing, model design, training, and evaluation. Two architectures are implemented: a custom CNN and ResNet18, both built using PyTorch. The pipeline includes:

1. Data preprocessing and augmentation
2. Model architecture design

3. Training with optimized hyperparameters
4. Evaluation using accuracy, precision, recall, F1-score, and ROC metrics

4 Data Preprocessing

The preprocessing pipeline includes:

1. **Download**: Fetch dataset via Kaggle API.
2. **Extraction**: Unzip into train/val/test folders.
3. **Re-split**: Merge and split into 80/10/10 ratio.
4. **Resize**: Scale images to 224×224.
5. **Normalization**: Scale pixel values to [0, 1].
6. **Augmentation**: Apply random rotations, flips, and brightness adjustments to training data.
7. **Conversion**: Transform images to PyTorch tensors.

The script `src/preprocess.py` automates these steps.

5 Model Architectures

5.1 Custom CNN

The custom CNN consists of:

- 3 convolutional layers (16, 32, 64 filters, 3×3 kernels)
- ReLU activation, max-pooling (2×2)
- 2 fully connected layers (512 units, 2 units for output)
- Dropout (0.5) for regularization

5.2 ResNet18

ResNet18, a pre-trained model from PyTorch's `torchvision`, is fine-tuned by:

- Replacing the final fully connected layer for binary classification
- Freezing initial layers and training the last block

6 Training & Hyperparameters

Both models are trained using:

- **Loss Function**: Binary Cross-Entropy with logits

- **Optimizer**: Adam (learning rate = 0.001)
- **Batch Size**: 32
- **Epochs**: 20
- **Class Weighting**: Adjusted for imbalance (2.4× for NORMAL)
- **Learning Rate Scheduler**: ReduceLROnPlateau (factor=0.5, patience=3)

Training is performed on a GPU-enabled PyTorch environment.

7 Evaluation Metrics

Models are evaluated using:

- **Accuracy**: Proportion of correct predictions
- **Precision**: True positives / (True positives + False positives)
- **Recall**: True positives / (True positives + False negatives)
- **F1-Score**: Harmonic mean of precision and recall
- **ROC-AUC**: Area under the Receiver Operating Characteristic curve

8 Results & Discussion

8.1 Accuracy and Loss

For the custom CNN, training over 20 epochs shows a steady decrease in train loss from 0.2838 to 0.0688, with train accuracy improving from 88.26% to 97.42%. Validation accuracy peaks at 96.58% (epoch 17, validation loss 0.0790), though fluctuations in validation loss (e.g., 0.9358 at epoch 16) indicate some overfitting. For ResNet18, train loss decreases from 0.4035 to 0.2813, with train accuracy rising from 82.45% to 88.41%. Validation accuracy peaks at 93.16% (epoch 16, validation loss 0.1904), with more stable validation performance compared to the CNN.

8.2 Classification Performance

On the test set, the custom CNN achieves:

- Accuracy: 97.96%
- Precision: 98.00%
- Recall: 97.96%
- F1-Score: 97.97%
- AUC: 99.69%

ResNet18 achieves:

- Accuracy: 92.67%
- Precision: 93.01%
- Recall: 92.67%
- F1-Score: 92.77%
- AUC: 97.49%

The custom CNN demonstrates superior performance across all metrics, particularly in precision and AUC, indicating excellent discrimination between NORMAL and PNEUMONIA classes.

8.3 Comparison between CNN & ResNet

Contrary to expectations, the custom CNN outperforms ResNet18, achieving a test accuracy of 97.96% compared to ResNet18's 92.67%. The CNN's simpler architecture may better suit the dataset's characteristics, avoiding overfitting seen in ResNet18's deeper structure. ResNet18's pre-trained weights, while effective, may not fully adapt to the specific features of this dataset, leading to lower performance despite faster per-epoch training (4.93s vs. 26.10s for CNN).

9 Conclusion & Future Work

This project demonstrates the efficacy of deep learning for pneumonia detection, with the custom CNN outperforming ResNet18, achieving a test accuracy of 97.96%. The high AUC (99.69%) suggests robust classification capability. Future work includes:

- Exploring Transformer-based models for improved feature extraction
- Incorporating multi-modal data (e.g., patient metadata) to enhance predictions
- Addressing class imbalance with advanced techniques (e.g., GANs)

10 References

- Kermany, Daniel, et al. "Identifying Medical Diagnoses and Treatable Diseases by Image-Based Deep Learning." *Cell*, 2018.
- He, Kaiming, et al. "Deep Residual Learning for Image Recognition." *CVPR*, 2016.
- PyTorch Documentation. https://pytorch.org/docs/stable/index.html